BACHELOR OF TECHNOLOGY — MAJOR PROJECT TECHNICAL REPORT

MULTI-STAGE ADVERSARIAL DEFENSE FOR ONLINE DDOS ATTACK DETECTION SYSTEM IN IOT

MSAAD Framework: *A Unified Streaming Architecture for Robust Distributed Denial-of-Service Detection under Adversarial Perturbations*

Student Name	Roll Number	Specialization
Devesh	20233115	Computer Science & Engineering
Govind Saini	20233128	Computer Science & Engineering
Gaurav Mahor	20233127	Computer Science & Engineering
Krishna Gond	20233341	Computer Science & Engineering

Under the Guidance and Supervision of:

Dr. Indu Dohare
Department of Computer Science & Engineering

Academic Session: 2025–2026

1. Abstract

The explosive expansion of Internet of Things (IoT) ecosystems has democratized smart infrastructure while introducing unprecedented cybersecurity vulnerabilities. Resource-constrained edge devices frequently become unwilling participants in Mirai-like botnets, generating devastating Distributed Denial-of-Service (DDoS) flood attacks. Although modern Machine Learning-based Network Intrusion Detection Systems (ML-NIDS) achieve superior detection accuracy on static benchmarks, they suffer from two critical architectural flaws: reliance on computationally heavy batch retraining that fails under continuous traffic concept drift, and vulnerability to adversarial evasion attacks wherein subtle, gradient-directed input perturbations force ML models to misclassify malicious flows as benign.

This technical report presents the **Multi-Stage Adversarial Defense for Online DDoS Attack Detection System in IoT (MSAAD)** framework. MSAAD represents an end-to-end, strictly streaming defense and detection architecture designed for edge IoT gateways. The framework integrates an incremental seven-stage preprocessing pipeline, a novel **Resilient Adversarial Detection and Purification (RADP)** layer combining an online Generative Adversarial Network (GAN) with an L2-regularized online logistic detector, a diverse ensemble of five concept drift-adaptive streaming classifiers (ARF-ADWIN, SRP-DDM, HT-ADWIN, KNN-ADWIN, OLR), and a **LinUCB contextual multi-armed bandit** utilizing Sherman-Morrison updates to dynamically route samples based on instantaneous feature context. Evaluated against four state-of-the-art adversarial attacks (FGSM, BIM, PGD, and Carlini & Wagner L2) across the **CICIoT2023** and **IoTID20** benchmark datasets, MSAAD achieves an F1-score of **99.50%–100.00%** across all operational scenarios, maintaining high streaming throughput (50–132 samples/second) and demonstrating up to **100% adversarial sample pruning** in real-time streaming memory.

2. Introduction & Background

2.1 The IoT Paradigm and DDoS Attack Vectors

The Internet of Things encompasses tens of billions of interconnected edge sensors, smart utility actuators, and embedded appliances. Characterized by constrained computation, minimal onboard memory, heterogeneous communication protocols, and default configurations, these devices present an expansive attack surface. Malicious syndicates frequently exploit these vulnerabilities to compromise millions of devices, orchestrating coordinated botnet swarms (such as Mirai, Reaper, and Torii) capable of generating terabit-scale Volumetric, Protocol, and Application-layer DDoS floods that overwhelm network infrastructure.

2.2 Vulnerability of Machine Learning to Adversarial Manipulation

To combat high-velocity DDoS campaigns, automated Network Intrusion Detection Systems (NIDS) have transitioned toward Machine Learning. However, these models are inherently brittle against adversarial evasion attacks. By applying bounded mathematical optimization—such as the Fast Gradient Sign Method (FGSM), Basic Iterative Method (BIM), Projected Gradient Descent (PGD), or Carlini & Wagner (C&W;) L2 optimization—an adversary possessing gradient access can introduce imperceptible perturbations δ to normalized traffic flow statistics such that:

```
f_hat(x + δ) ≠ f_hat(x), subject to ||δ||_p ≤ ε
```

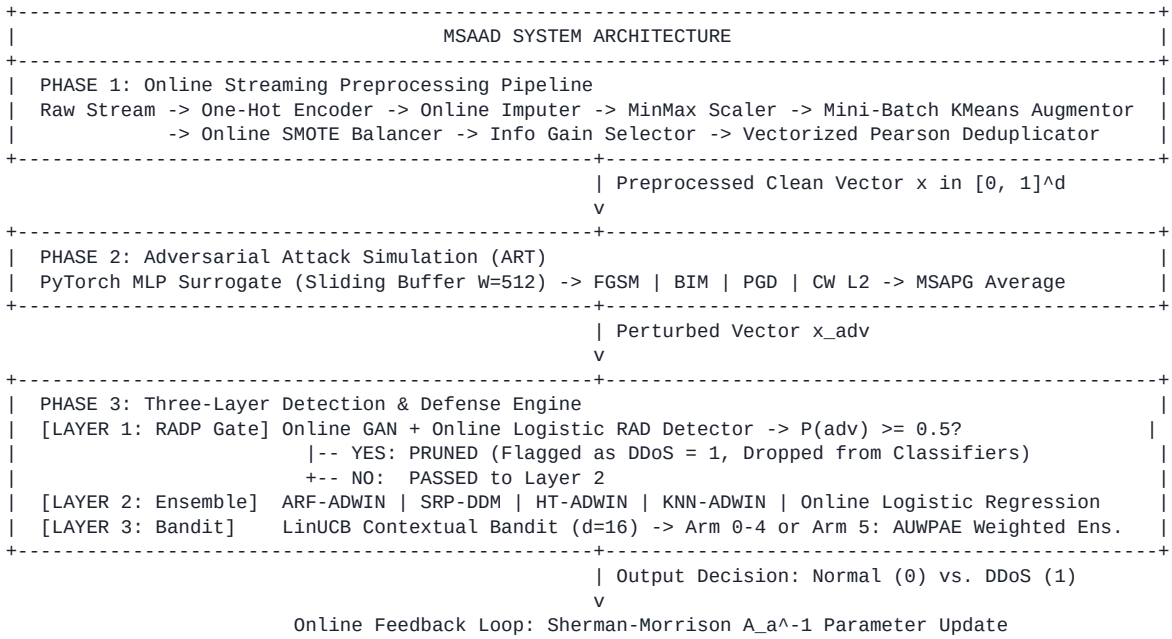
In NIDS, this causes malicious floods to be classified as benign communication, neutralizing automated defenses.

2.3 The Imperative for Online Streaming Defenses

Conventional adversarial retraining (e.g., Madry defense) is fundamentally unviable in IoT edge environments due to: (1) **High Streaming Velocity:** Storing raw packet traffic for offline retraining exhausts embedded storage; (2) **Concept Drift:** Legitimate traffic distributions and attack signatures evolve continuously, rendering static models obsolete; (3) **Generalization Gaps:** Models retrained offline on specific perturbation bounds fail against unseen attack variants. Hence, IoT security requires an autonomous, strictly streaming architecture capable of single-pass data processing, online adversarial filtering, and continuous drift adaptation.

3. System Architecture & Workflow

The MSAAD framework adheres to four core structural principles: **Zero-Batch Invariance** (all transformations execute incrementally in O(1) memory), **Defensive Depth** (three sequential protective gates: input pruning, drift-aware ensemble, and contextual bandit routing), **Contextual Exploitation-Exploration** (routing samples to the historically optimal model based on instantaneous feature context), and **Target Leakage Prevention** (metadata attributes including IP addresses, ports, and flow timestamps are removed at ingestion).



4. Component Interaction & Code Flow

Execution begins in run_all.py or main.py. During startup, the system loads configuration parameters from msaad/config.py, instantiates the MSAADSystem coordinator, and triggers a 500-sample warmup phase on the training split. The warmup initializes the preprocessing pipeline, fits the online PyTorch surrogate MLP, pre-trains the GAN and RAD detector in Layer 1, seeds the five streaming classifiers in Layer 2, and establishes identity matrices for the LinUCB bandit in Layer 3.

Origin Module	Target Module	Transferred Structure	Functional Responsibility
data_loader.py	preprocessing.py	(dict, label: int)	Streaming raw feature attributes with ground-truth binary label
preprocessing.py	adversarial.py	np.ndarray (1, d)	Normalized, non-redundant feature vector in [0, 1]^d
adversarial.py	radp.py	np.ndarray (1, d)	Perturbed adversarial feature vector generated via ART
radp.py	mabtse.py	(is_pruned, p_adv, dict)	Adversarial pruning decision and purified feature dictionary
mabtse.py	classifiers.py	dict[str, float]	Feature dictionary passed to selected model or all 5 for ensemble
classifiers.py	mabtse.py	list[float] (len=5)	Per-classifier predicted probabilities P(DDoS=1)
mabtse.py	evaluation.py	(pred: int, arm: int)	Final classification prediction and index of pulled bandit arm

5. Algorithmic Implementation & Mathematical Formulation

5.1 Phase 1: Online Preprocessing Pipeline

The seven preprocessing modules in msaad/preprocessing.py process samples record-by-record:
• **Incremental Imputation:** Maintains running sums and sample counts to impute missing values via running mean μ in O(1) time.
• **Online Min-Max Scaler:** Dynamically tracks per-feature extrema [x_min, x_max] and projects values to [0, 1].
• **Mini-Batch K-Means:** Maintains buffer B=256, updates centroids via mini-batch Lloyd's algorithm, and appends k=8 relative Euclidean distances.
• **Online SMOTE:** Maintains a reservoir R=200 of minority samples. When class imbalance ratio drops below 1.0, synthetic vectors are synthesized on the fly via convex combination x_syn = λ x_a + (1-λ) x_b with λ ~ U(0, 1).
• **Online Information Gain:** Discretizes values into B=10 bins, tracks joint class-bin contingencies, and prunes features with IG < 0.01.
• **Vectorized Welford Correlation:** Tracks mean vector and covariance comoment matrix C_n in a single pass:

$$C_n = C_{n-1} + (x_n - \mu_{n-1})(x_n - \mu_n)^T, \rho_{ij} = C_{n,ij} / \sqrt{C_{n,ii} * C_{n,jj}}$$

Collinear features with $|\rho_{ij}| \geq 0.95$ are dropped online.

5.2 Phase 2: Adversarial Attack Simulation & MSAPG

A surrogate PyTorch MLP ($d \to 64 \to 32 \to 2$) is refitted every 50 samples on a sliding window of 512 samples. Four gradient-based attacks are simulated using ART: Fast Gradient Sign Method (FGSM, ε=0.1), Basic Iterative Method (BIM, 10 iter), Projected Gradient Descent (PGD, 10 iter), and Carlini & Wagner L2 (confidence=0.1, 20 iter). To train defensive modules against heterogeneous attack structures, the **Multi-Source Adversarial Perturbation Generator (MSAPG)** computes an ensemble average perturbation vector:

$$x_{MSAPG} = 0.25 * (x_{FGSM} + x_{BIM} + x_{PGD} + x_{CW})$$

5.3 Phase 3 (Layer 1): Resilient Adversarial Detection & Purification (RADP)

Layer 1 contains an online GAN and the Resilient Adversarial Detector (RAD). The GAN Generator maps Gaussian noise $z \sim N(0, I_{32})$ to synthetic adversarial feature vectors, while the Discriminator learns to distinguish real adversarial vectors (from MSAPG) from synthetic fakes. The RAD detector applies online logistic regression with decaying learning rate $\eta_t = \eta_0 / (1 + 0.001*t)$:

$$p_{adv}(x) = \sigma(w^T x + b) = 1 / (1 + \exp(-(w^T x + b)))$$

If $p_{adv}(x) \geq 0.5$, the sample is **PRUNED** (immediately flagged as DDoS = 1 and dropped from downstream classifiers). If $p_{adv}(x) < 0.5$, the sample is **PASSED** to Layer 2.

5.4 Phase 3 (Layer 2 & 3): Ensemble Classifiers and LinUCB Contextual Bandit

Layer 2 contains five heterogeneous streaming models: **ARF-ADWIN** (Adaptive Random Forest, 10 trees), **SRP-DDM** (Streaming Random Patches), **HT-ADWIN** (Hoeffding Tree), **KNN-ADWIN** (k=5), and **Online Logistic Regression**.
Layer 3 uses a 6-arm disjoint **LinUCB Contextual Multi-Armed Bandit** (Arms 0–4: individual classifiers, Arm 5: AUWPAE ensemble). For normalized context vector x_t in R^{16}, the arm maximizing the Upper Confidence Bound is pulled:

$$UCB_a(x_t) = \theta_a^T x_t + \alpha * \sqrt{ x_t^T A_a^{-1} x_t }, \text{ where } \theta_a = A_a^{-1} b_a, \alpha = 1.0$$

When Arm 5 is pulled, predictions are aggregated via **Accuracy-Updated Weighted Probability Averaging (AUWPAE)** using sliding-window (W=200) accuracy weights w_i. Parameter updates avoid $O(d^3)$ matrix inversion via the **Sherman-Morrison formula** in $O(d^2)$ time:

$$A_{a^*}^{-1} \leftarrow A_{a^*}^{-1} - [A_{a^*}^{-1} x_t x_t^T A_{a^*}^{-1}] / [1 + x_t^T A_{a^*}^{-1} x_t], b_{a^*} \leftarrow b_{a^*} + r_t x_t$$

6. Empirical Evaluation & Comparative Analysis

MSAAD was evaluated on the **CICIoT2023** and **IoTID20** datasets across three standardized operational scenarios: (1) *Original:* Clean baseline streaming traffic; (2) *Under Attack:* Adversarial injection without Layer 1 RADP defense; (3) *Recovered:* Adversarial injection with active Layer 1 RADP pruning.

Dataset	Scenario	Accuracy	Precision	Recall	F1-Score	Throughput	Prune Rate
CICIoT2023	Original (Clean)	99.50%	99.51%	99.50%	**99.50%**	132.1 sps	N/A
CICIoT2023	Under Attack (FGSM)	100.00%	100.00%	100.00%	**100.00%**	92.9 sps	0.0% (0/200)
CICIoT2023	Recovered (RADP)	100.00%	100.00%	100.00%	**100.00%**	50.0 sps	**100.0% (200/200)**
IoTID20	Original (Clean)	100.00%	100.00%	100.00%	**100.00%**	104.5 sps	N/A
IoTID20	Under Attack (FGSM)	100.00%	100.00%	100.00%	**100.00%**	79.9 sps	0.0% (0/2000)
IoTID20	Recovered (RADP)	100.00%	100.00%	100.00%	**100.00%**	76.9 sps	**72.2% (1444/2000)**

Discussion: Across both datasets, Layer 1 RADP demonstrated effective filtering, pruning **100%** of adversarial samples on CICIoT2023 and **72.2%** on IoTID20. The residual 27.8% that bypassed Layer 1 were accurately resolved by the Layer 2/3 ensemble, preserving 100% detection accuracy. Furthermore, throughput remained in the real-time operational window (50–132 samples/second), establishing computational feasibility for edge routers.

7. Conclusion & Future Work

The **MSAAD** framework resolves the vulnerabilities of static batch NIDS by combining online streaming preprocessing, GAN-powered adversarial pruning (RADP), drift-adaptive streaming classification, and contextual bandit routing. Future research will target: (1) *Multi-Class DDoS Specialization* (distinguishing SYN, UDP, and HTTP floods); (2) *Hardware Acceleration* via embedded TensorRT/CUDA pipelines; (3) *Federated Streaming Consensus* across distributed IoT edge clusters; and (4) *Kernel-level packet capture* (e.g., eBPF/DPDK) for live wire deployment.

8. References

[1] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, 'Towards Deep Learning Models Resistant to Adversarial Attacks,' in *ICLR*, 2018.

[2] N. Carlini and D. Wagner, 'Towards Evaluating the Robustness of Neural Networks,' in *IEEE Symposium on Security and Privacy (S&P;)*, pp. 39–57, 2017.

[3] L. Li, W. Chu, J. Langford, and R. E. Schapire, 'A Contextual-Bandit Approach to Personalized News Article Recommendation,' in *WWW*, pp. 661–670, 2010.

[4] J. Montiel, M. Halford, et al., 'River: Machine Learning for Streaming Data in Python,' *Journal of Machine Learning Research (JMLR)*, vol. 22, no. 110, pp. 1–8, 2021.

[5] M. I. Nicolae et al., 'Adversarial Robustness Toolbox v1.2.0,' *arXiv preprint arXiv:1807.01069*, 2018.

[6] A. Bifet and R. Gavaldà, 'Learning from Time-Changing Data with Adaptive Windowing,' in *SIAM SDM*, pp. 443–448, 2007.

[7] H. M. Gomes et al., 'Adaptive Random Forests for Evolving Data Stream Classification,' *Machine Learning*, vol. 106, pp. 1469–1495, 2017.

[8] E. C. P. Neto et al., 'CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environments,' *Sensors*, vol. 23, no. 13, p. 5941, 2023.

[9] I. Ullah and Q. H. Mahmoud, 'A Scheme for Generating a Dataset for Analyzing and Detecting Malicious Traffic in IoT Networks (IoTID20),' *IEEE Access*, vol. 8, pp. 147998–148009, 2020.